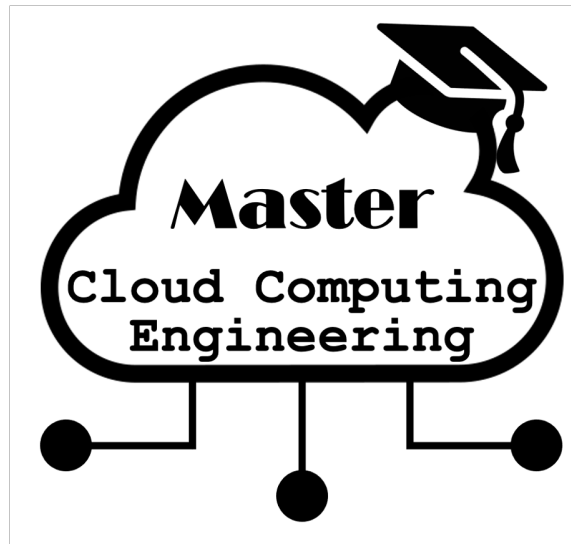




## DevOps PT - Assignment Release/Deploy/Operate - Mika

Hochschule Burgenland  
Studiengang MCCE  
Sommersemester 2025

Harald Beier\*    Susanne Peer<sup>†</sup>    Patrick Prugger<sup>‡</sup>    Philipp Palatin<sup>§</sup>

23. Juni 2025

---

\*2410781028@hochschule-burgenland.at

<sup>†</sup>2410781002@hochschule-burgenland.at

<sup>‡</sup>2410781029@hochschule-burgenland.at

<sup>§</sup>2310781027@hochschule-burgenland.at

# Inhaltsverzeichnis

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                             | <b>3</b>  |
| <b>2</b> | <b>Architekturüberblick</b>                   | <b>3</b>  |
| 2.1      | Hochebenen-Architektur . . . . .              | 3         |
| 2.2      | Datenfluss und Interaktionen . . . . .        | 4         |
| <b>3</b> | <b>AWS Services und Komponenten</b>           | <b>5</b>  |
| 3.1      | Frontend-Layer . . . . .                      | 5         |
| 3.1.1    | Amazon S3 + CloudFront . . . . .              | 5         |
| 3.2      | API-Layer . . . . .                           | 5         |
| 3.2.1    | Amazon API Gateway . . . . .                  | 5         |
| 3.3      | Computing-Layer . . . . .                     | 5         |
| 3.3.1    | AWS Lambda Functions . . . . .                | 5         |
| 3.4      | Datenbank-Layer . . . . .                     | 6         |
| 3.4.1    | Amazon DynamoDB . . . . .                     | 6         |
| <b>4</b> | <b>Infrastructure-as-Code (IaC) Strategie</b> | <b>6</b>  |
| 4.1      | Terraform als Provisionierungstool . . . . .  | 6         |
| 4.2      | Terraform-Projektstruktur . . . . .           | 7         |
| 4.3      | Beispiel Terraform-Code . . . . .             | 8         |
| <b>5</b> | <b>Automatisierungskonzept</b>                | <b>9</b>  |
| 5.1      | CI/CD Pipeline mit GitHub Actions . . . . .   | 9         |
| 5.1.1    | Pipeline-Stages . . . . .                     | 9         |
| 5.2      | GitHub Actions Workflow . . . . .             | 9         |
| <b>6</b> | <b>Sicherheit und Authentifizierung</b>       | <b>10</b> |
| 6.1      | AWS-Authentifizierung für Terraform . . . . . | 10        |
| 6.1.1    | IAM Roles mit OIDC (Empfohlen) . . . . .      | 10        |
| 6.2      | Anwendungssicherheit . . . . .                | 11        |
| 6.2.1    | Lambda Function Security . . . . .            | 11        |
| 6.2.2    | API Gateway Security . . . . .                | 11        |
| 6.2.3    | DynamoDB Security . . . . .                   | 11        |
| <b>7</b> | <b>Kosten-Optimierung</b>                     | <b>11</b> |
| 7.1      | Pricing-Modell . . . . .                      | 11        |
| 7.2      | Kostenoptimierung-Strategien . . . . .        | 11        |
| <b>8</b> | <b>Fazit</b>                                  | <b>12</b> |
|          | <b>Literaturverzeichnis</b>                   | <b>13</b> |

# 1 Einleitung

Diese Konzeption beschreibt die automatische Provisionierung einer vollständig serverlosen Todo-Anwendung in der AWS Cloud [Jain and Sharma, 2021][Wen et al., 2023]. Die Lösung nutzt moderne Infrastructure-as-Code (IaC) Prinzipien mit Terraform als primärem Automatisierungstool[Özdoğan et al., 2023][HashiCorp, 2023] und implementiert eine CI/CD-Pipeline für kontinuierliche Bereitstellung.

## **Kernkomponenten:**

- **Frontend:** Angular SPA gehostet auf Amazon S3 mit CloudFront CDN [clo, 2012][Tyagi, 2025]
- **Backend:** AWS Lambda Functions für CRUD-Operationen [Hussain, 2024][Tanneru, 2024]
- **Datenbank:** Amazon DynamoDB für hochperformante NoSQL-Datenspeicherung [Haseeb and Pattun, 2017][Ahmed, 2024]
- **API Gateway:** RESTful API-Endpunkte mit integrierter Authentifizierung
- **Authentifizierung:** RESTful API-Endpunkte mit integrierter Authentifizierung

**Automatisierung:** Vollständige Infrastructure-as-Code mit Terraform, GitHub Actions CI/CD Pipeline, und automatisierte Tests.

# 2 Architekturüberblick

## 2.1 Hochebenen-Architektur

Die Architektur der Todo-Anwendung basiert auf einem serverlosen Ansatz, der eine hohe Skalierbarkeit und Kosteneffizienz bietet. Die Anwendung besteht aus mehreren AWS-Diensten, die nahtlos integriert sind, um eine robuste und wartungsarme Lösung zu schaffen. Die folgende Abbildung zeigt die Architekturkomponenten und deren Interaktionen [Maharjan, 2022].

## AWS Cloud Environment

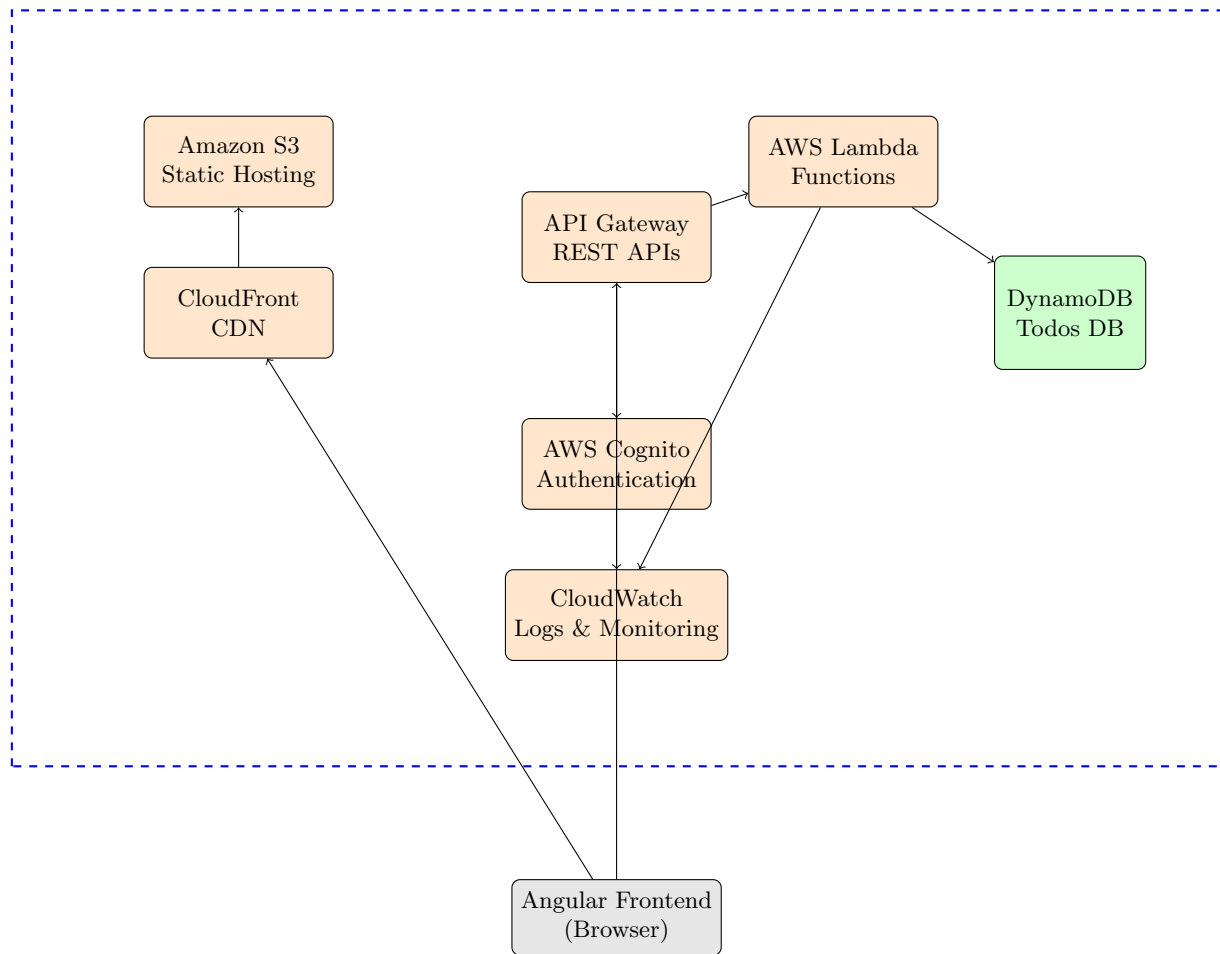


Abbildung 1: AWS Serverless Architecture Diagramm

### 2.2 Datenfluss und Interaktionen

1. **Client Request:** Browser lädt Angular App von S3 über CloudFront
2. **API Calls:** Frontend sendet HTTP-Requests an API Gateway
3. **Authentication:** Cognito validiert Benutzer-Tokens
4. **Processing:** Lambda Functions verarbeiten Business Logic
5. **Data Persistence:** DynamoDB speichert/liest Todo-Daten
6. **Response:** JSON-Antworten zurück zum Frontend

## 3 AWS Services und Komponenten

### 3.1 Frontend-Layer

#### 3.1.1 Amazon S3 + CloudFront

- **S3 Bucket:** Static Website Hosting für Angular-Bundle [Infosys, 2025]
- **CloudFront Distribution:** Globale CDN für niedrige Latenz
- **Route 53:** DNS-Management (optional für Custom Domain)

**Vorteile:**

- Skalierbarkeit ohne Server-Management
- Globale Verfügbarkeit durch Edge Locations [Xu, 2024]
- Kosteneffizient für statische Inhalte

### 3.2 API-Layer

#### 3.2.1 Amazon API Gateway

- RESTful API-Endpunkte
- Request/Response Transformation
- Throttling und Rate Limiting
- CORS-Konfiguration
- Integration mit AWS Lambda

**Endpunkt-Struktur:**

| HTTP Method | Endpoint                         |
|-------------|----------------------------------|
| GET         | /todos - Alle Todos abrufen      |
| POST        | /todos - Neues Todo erstellen    |
| PUT         | /todos/{id} - Todo aktualisieren |
| DELETE      | /todos/{id} - Todo löschen       |

Tabelle 1: REST API Endpunkte

### 3.3 Computing-Layer

#### 3.3.1 AWS Lambda Functions

- **Get-Todos Function:** Liest alle Todos aus DynamoDB
- **Create-Todo Function:** Erstellt neues Todo-Item

- **Update-Todo Function:** Aktualisiert bestehende Todos
- **Delete-Todo Function:** Entfernt Todo-Items

**Konfiguration:**

- **Runtime:** Node.js 18.x oder Python 3.11
- **Memory:** 128-512 MB je nach Funktion
- **Timeout:** 30 Sekunden maximum

## 3.4 Datenbank-Layer

### 3.4.1 Amazon DynamoDB

| Attribut      | Wert                                                  |
|---------------|-------------------------------------------------------|
| Table Name    | TodosTable                                            |
| Partition Key | userId (String)                                       |
| Sort Key      | todoId (String)                                       |
| Attributes    | title, description, completed<br>createdAt, updatedAt |
| Skalierung    | On-Demand Billing                                     |

Tabelle 2: DynamoDB Tabellen-Konfiguration

## 4 Infrastructure-as-Code (IaC) Strategie

### 4.1 Terraform als Provisionierungstool

Bei der Auswahl eines geeigneten Infrastructure-as-Code (IaC) Tools stehen verschiedene Optionen zur Verfügung, die jeweils unterschiedliche Stärken und Schwächen aufweisen. Im Folgenden werden die wichtigsten Alternativen zu Terraform analysiert und deren charakteristische Eigenschaften gegenübergestellt. Die Bewertung der verschiedenen Tools erfolgt anhand mehrerer kritischer Dimensionen, die für die erfolgreiche Implementierung von Infrastructure-as-Code-Projekten entscheidend sind.

- **AWS CloudFormation** stellt als nativer AWS-Service eine solide Grundlage für AWS-spezifische Infrastrukturen dar. Die enge Integration in das AWS-Ökosystem bietet Vorteile bei der Verwaltung von AWS-Ressourcen, jedoch beschränkt sich die Anwendbarkeit ausschließlich auf die Amazon-Cloud-Plattform. Die JSON- oder YAML-basierte Syntax kann bei komplexeren Infrastrukturen schnell unübersichtlich werden.
- **Terraform** ist ein Open-Source-Tool, das eine deklarative Syntax in HashiCorp Configuration Language (HCL) verwendet. Es bietet eine starke Multi-Cloud-Unterstützung und eine große Community, die kontinuierlich zur Weiterentwicklung beiträgt. Die explizite Verwaltung des Zustands ermöglicht eine präzise Kontrolle über die Infrastruktur, was insbesondere bei komplexen Umgebungen von Vorteil ist.

| Kriterium        | Terraform   | AWS CloudFormation | AWS CDK          | Ansible   |
|------------------|-------------|--------------------|------------------|-----------|
| Multi-Cloud      | ✓           | ×                  | ×                | ✓         |
| Lernkurve        | ●           | ●                  | ▲ Schwer         | ✓ Einfach |
| Community        | ✓ Sehr groß | ●                  | ▲ Klein          | ✓ Groß    |
| Syntax           | ✓ HCL       | ▲ JSON/YAML        | ▲ Code           | ✓ YAML    |
| State Management | ✓ Explizit  | ✓ Implizit         | ✓ CloudFormation | ▲ Schwach |

Tabelle 3: Vergleichsmatrix der Infrastructure-as-Code Tools

- **AWS CDK (Cloud Development Kit)** ermöglicht die Definition von Infrastruktur mittels vertrauter Programmiersprachen wie TypeScript, Python oder Java. Während dies für Entwickler attraktiv erscheint, erhöht sich dadurch die Komplexität erheblich, da sowohl die jeweilige Programmiersprache als auch die CDK-spezifischen Konzepte beherrscht werden müssen.
- **Ansible** zeichnet sich durch seine einfache YAML-Syntax und die niedrige Einstiegshürde aus. Als primäres Konfigurationsmanagement-Tool konzipiert, weist es jedoch Schwächen im Bereich des State Managements auf, was bei der Verwaltung komplexer Infrastrukturen problematisch werden kann.

Die Entscheidung für Terraform als primäres Provisionierungstool basiert auf der Kombination aus Multi-Cloud-Fähigkeit, starker Community-Unterstützung und der expliziten Verwaltung des Zustands. Diese Eigenschaften ermöglichen eine flexible, skalierbare und wartungsfreundliche Infrastruktur, die den Anforderungen moderner Cloud-Anwendungen gerecht wird.

## 4.2 Terraform-Projektstruktur

Die Projektstruktur für die Terraform-Konfiguration folgt den Best Practices für eine modulare und wartbare Architektur. Die Hauptkonfigurationsdateien befinden sich im Wurzelverzeichnis, während spezifische Module und Umgebungen in separaten Verzeichnissen organisiert sind. Diese Struktur ermöglicht eine klare Trennung der Verantwortlichkeiten und erleichtert die Wiederverwendbarkeit von Code.

```

1 terraform/
2 |-- main.tf           # Hauptkonfiguration
3 |-- variables.tf      # Input-Variablen
4 |-- outputs.tf        # Output-Werte
5 |-- terraform.tfvars  # Umgebungsspezifische Werte
6 |-- backend.tf        # Remote State Configuration
7 |-- modules/
8 |   |-- api-gateway/  # API Gateway Module
9 |   |-- lambda/       # Lambda Functions Module
10 |   |-- dynamodb/    # DynamoDB Module

```

```

11 | |-- s3-cloudfront/      # Frontend Hosting Module
12 | |-- cognito/          # Authentication Module
13 |-- environments/
14 |   |-- dev/
15 |   |-- staging/
16 |   |-- production/

```

Listing 1: Terraform Projektstruktur

### 4.3 Beispiel Terraform-Code

Die Hauptkonfiguration für die Todo-Anwendung umfasst die Definition der AWS-Provider, die Konfiguration der Ressourcen und die Integration der Module. Die folgenden Code-Snippets zeigen die wichtigsten Teile der Terraform-Konfiguration, einschließlich der Definition von Variablen, Modulen und Ressourcen.

```

1 # main.tf
2 terraform {
3   required_version = ">= 1.5"
4   required_providers {
5     aws = {
6       source  = "hashicorp/aws"
7       version = "~> 5.0"
8     }
9   }
10 }
11
12 provider "aws" {
13   region = var.aws_region
14
15   default_tags {
16     tags = {
17       Project      = "TodoApp"
18       Environment  = var.environment
19       ManagedBy    = "Terraform"
20     }
21   }
22 }
23
24 # DynamoDB Table
25 module "dynamodb" {
26   source = "./modules/dynamodb"
27
28   table_name = "${var.project_name}-todos-${var.environment}"
29   environment = var.environment
30 }
31
32 # Lambda Functions
33 module "lambda_functions" {
34   source = "./modules/lambda"
35
36   environment      = var.environment
37   dynamodb_table  = module.dynamodb.table_name
38   dynamodb_arn     = module.dynamodb.table_arn
39 }
40

```



```

41 # API Gateway
42 module "api_gateway" {
43     source = "../modules/api-gateway"
44
45     environment      = var.environment
46     lambda_functions = module.lambda_functions.function_arns
47     cognito_user_pool_arn = module.cognito.user_pool_arn
48 }

```

Listing 2: Hauptkonfiguration (main.tf)

## 5 Automatisierungskonzept

### 5.1 CI/CD Pipeline mit GitHub Actions

#### 5.1.1 Pipeline-Stages

1. **Code Commit:** Developer pushed Code zu GitHub
2. **Validation:** Terraform fmt, validate, security scanning
3. **Planning:** Terraform plan für Infrastructure Changes
4. **Testing:** Unit Tests für Lambda Functions
5. **Deployment:** Terraform apply (umgebungsabhängig)
6. **Frontend Build:** Angular Build und S3 Upload
7. **Integration Tests:** End-to-End Tests der API
8. **Monitoring:** Health Checks und Alerting

### 5.2 GitHub Actions Workflow

```

1 # .github/workflows/deploy.yml
2 name: Deploy Todo App Infrastructure
3
4 on:
5   push:
6     branches: [main, develop]
7   pull_request:
8     branches: [main]
9
10 env:
11   AWS_REGION: eu-central-1
12   TF_VERSION: 1.5.7
13
14 jobs:
15   terraform:
16     name: Terraform Plan/Apply
17     runs-on: ubuntu-latest
18

```

```

19  steps:
20  - name: Checkout code
21    uses: actions/checkout@v4
22
23  - name: Setup Terraform
24    uses: hashicorp/setup-terraform@v2
25    with:
26      terraform_version: ${ env.TF_VERSION }
27
28  - name: Configure AWS credentials
29    uses: aws-actions/configure-aws-credentials@v2
30    with:
31      role-to-assume: ${ secrets.AWS_ROLE_ARN }
32      aws-region: ${ env.AWS_REGION }
33
34  - name: Terraform Init
35    run: terraform init
36    working-directory: ./terraform
37
38  - name: Terraform Plan
39    run: terraform plan -no-color
40    working-directory: ./terraform
41
42  - name: Terraform Apply
43    if: github.ref == 'refs/heads/main'
44    run: terraform apply -auto-approve
45    working-directory: ./terraform

```

Listing 3: GitHub Actions Deployment Workflow

## 6 Sicherheit und Authentifizierung

### 6.1 AWS-Authentifizierung für Terraform

#### 6.1.1 IAM Roles mit OIDC (Empfohlen)

```

1  - name: Configure AWS credentials
2    uses: aws-actions/configure-aws-credentials@v2
3    with:
4      role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
5      role-session-name: GitHubActions-ToDoApp
6      aws-region: eu-central-1

```

Listing 4: AWS Credential Configuration

#### Vorteile:

- Keine langlebigen AWS Access Keys
- Automatische Token-Rotation
- Granulare Berechtigungen pro Repository

## 6.2 Anwendungssicherheit

### 6.2.1 Lambda Function Security

- VPC-Integration für Netzwerk-Isolation
- Environment Variables für Konfiguration
- AWS X-Ray für Tracing und Monitoring
- Least Privilege IAM Roles

### 6.2.2 API Gateway Security

- Cognito Authorizer für alle Endpunkte
- Request/Response Validation
- Rate Limiting (1000 requests/minute)
- CORS-Konfiguration

### 6.2.3 DynamoDB Security

- Encryption at Rest aktiviert
- Encryption in Transit (TLS)
- Point-in-Time Recovery
- VPC Endpoints für private Zugriffe

## 7 Kosten-Optimierung

### 7.1 Pricing-Modell

| Service            | Development | Production   |
|--------------------|-------------|--------------|
| Lambda             | \$5         | \$50         |
| API Gateway        | \$3         | \$35         |
| DynamoDB           | \$2         | \$25         |
| S3                 | \$1         | \$10         |
| CloudFront         | \$5         | \$50         |
| <b>Total/Monat</b> | <b>\$16</b> | <b>\$170</b> |

Tabelle 4: Monatliche Kostenschätzung (USD)

### 7.2 Kostenoptimierung-Strategien

1. **DynamoDB On-Demand:** Für unvorhersagbare Workloads

2. **Lambda Provisioned Concurrency:** Nur für kritische Funktionen
3. **S3 Intelligent Tiering:** Automatische Kostenoptimierung
4. **CloudFront Caching:** Reduzierte Origin-Requests
5. **Reserved Capacity:** Für vorhersagbare Workloads

## 8 Fazit

Abschließend lässt sich festhalten, dass die serverlose Todo-Anwendung auf AWS eine moderne, skalierbare und wartungsarme Lösung darstellt. Die Verwendung von Infrastructure-as-Code mit Terraform ermöglicht eine effiziente Verwaltung der Infrastruktur und eine nahtlose Integration in CI/CD-Pipelines. Folgende Best Practices wurden implementiert, um eine robuste und sichere Anwendung zu gewährleisten:

1. **Infrastructure as Code:** Alle Änderungen über Git versioniert
2. **Immutable Infrastructure:** Keine manuellen Änderungen in AWS Console
3. **Security by Design:** Least Privilege Principle durchgehend angewendet
4. **Cost Awareness:** Regelmäßige Cost Reviews und Budget Alerts
5. **Disaster Recovery:** Automatisierte Backups und Rollback-Prozesse

## Literaturverzeichnis

- [clo, 2012] (2012). Performance analysis of content delivery network (cloudfront) based on amazon cloud infrastructure. Master’s thesis, California State University.
- [Ahmed et al., 2018] Ahmed, M. R., Khatun, M. A., Ali, M. A., and Sundaraj, K. (2018). A literature review on nosql database for big data processing. *International Journal of Engineering & Technology*.
- [Haseeb and Pattun, 2017] Haseeb, A. and Pattun, G. (2017). A review on nosql: Applications and challenges. *International Journal of Advanced Research in Computer Science*, 8:203–207.
- [HashiCorp, 2024] HashiCorp (2024). Configuration language - terraform. <https://developer.hashicorp.com/terraform/language>. Accessed: 2025-06-23.
- [Hussain, 2024] Hussain, I. (2024). Performance analysis and optimization of amazon web service lambda functions in serverless systems. *International Journal of Scientific Research*, 13(11).
- [Infosys, 2025] Infosys (2025). Hosting static/single page web application on aws s3. <https://blogs.infosys.com/digital-experience/web-ui-ux/hosting-static-single-page-web-application-on-aws-s3.html>. Accessed: 2025-06-23.
- [Jain and Sharma, 2021] Jain, N. and Sharma, N. (2021). Serverless computing and aws lambda. *International Journal of Food and Agricultural Sciences*, 10(4):1126–1135.
- [Maharjan, 2022] Maharjan, C. (2022). Evaluating serverless computing. Master’s thesis, Louisiana State University.
- [Tanneru, 2024] Tanneru, B. (2024). Serverless computing with aws lambda: Evaluating suitability and scalability. *Journal of Software Engineering and Simulation*, 10(10):71–73.
- [Tyagi, 2025] Tyagi, A. (2025). Optimizing digital experiences with content delivery networks: Architectures, performance strategies, and future trends. *arXiv preprint arXiv:2501.06428*.
- [Wen et al., 2023] Wen, J., Chen, Z., Jin, X., and Liu, X. (2023). Rise of the planet of serverless computing: A systematic review. *ACM Computing Surveys*.
- [Xu, 2024] Xu, Y. (2024). Understanding and characterizing cdn services and paid features. Master’s thesis, University of Twente.
- [Özdoğan et al., 2023] Özdoğan, E., Ceran, O., and Üstündağ, M. T. (2023). Systematic analysis of infrastructure as code technologies. *Gazi University Journal of Science Part A: Engineering and Innovation*, 10(4):452–471.